**Universidad del Valle de Guatemala**
*Facultad de Ingeniería*
*Base de datos 2, sección 20*

# Proyecto 01
# MongoDB

*Dilary Sahari Cruz Lopez, 231010*
*María José Girón Isidro, 23559*

*Leonardo Dufrey Mejía Mejía, 23648*

# Índice

Índice ..... 1
ETAPA 01 - Modelado de datos y diseño del proyecto ..... 2
ETAPA 02 - Elaboración del Backend ..... 9

# ETAPA 01 - Modelado de datos y diseño del proyecto

## Descripción del problema

Se requiere el desarrollo de un sistema de pedidos y reseñas de restaurantes, haciendo uso de MongoDB como base de datos orientada a los documentos que se manejarán.

El sistema deberá permitir:

1. Registrar restaurantes
2. Registrar usuarios
3. Administrar artículos del menú
4. Crear y gestionar órdenes (pedidos)
5. Crear y consultar reseñas
6. Ejecutar consultas analíticas mediante el Aggregation Framework

El objetivo principal del programa será modelar de manera adecuada la información, utilizando documentos embebidos y referenciados, aplicando índices, agregaciones y transacciones.

## Alcance

El sistema deberá:

Registrar y permitir

- Usuarios
- Restaurantes
- Artículos de menú asociados a un restaurante

Crear y gestionar

- Órdenes realizadas por usuarios
- Reseñas asociadas a restaurantes y opcionalmente a órdenes

Consultar información mediante

- Filtros
- Ordenamientos
- Proyecciones
- Paginación

Generar reportes mediante agregaciones como

- Restaurantes mejor calificados
- Platillos más vendidos

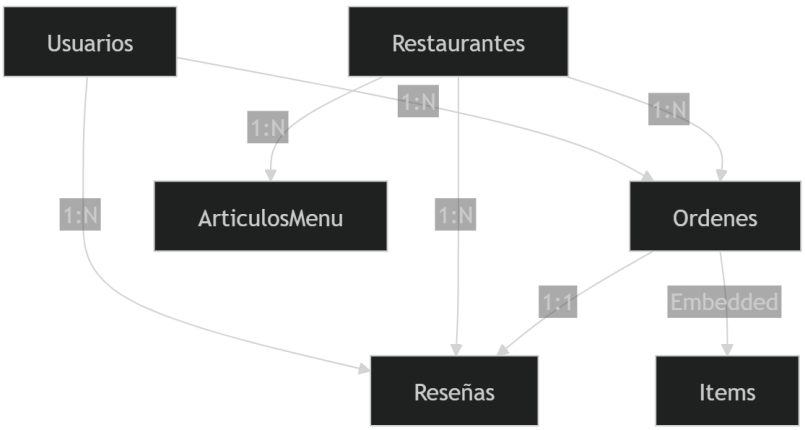
- Ventas por periodo

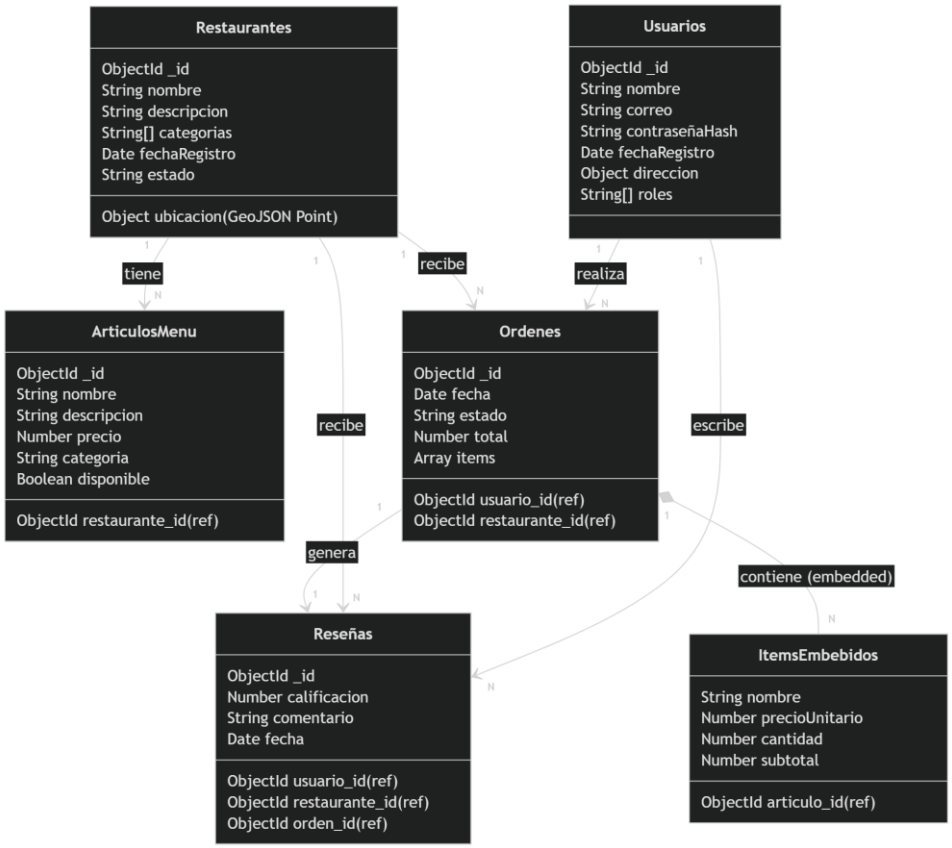
El sistema se tendrá que enfocar en la correcta gestión y modelado de datos, priorizando sobre todo el uso de documentos embebidos y referenciados, manejo de arrays, índices y transacciones.

## Supuestos

1. El sistema es un backend que expone funcionalidades mediante API
2. No se integran pasarelas reales de pago
3. Los restaurantes pueden tener múltiples artículos de menú.
4. Un usuario puede realizar múltiples pedidos
5. Un pedido puede contener múltiples artículos.
6. Una reseña puede asociarse a un restaurante y opcionalmente a un pedido específico.

## Modelo de datos detallado con diagramas de documentos.






## Justificación de embedding vs referencing.

Embedding

Se optó por embebar los items dentro de la colección Ordenes, ya que estos representan el estado histórico del pedido en el momento en que fue realizado. Cada ítem incluido en una orden contiene información como el nombre del artículo, precio unitario, cantidad y subtotal, lo cual permite preservar los datos tal como existían al momento de la compra.

Esta decisión es importante porque el precio o la descripción de un artículo en el menú pueden cambiar con el tiempo. Si únicamente se almacenara la referencia al artículo, cualquier modificación posterior afectaría la integridad histórica del pedido. Al embebar los datos relevantes dentro del documento de la orden, se garantiza la inmutabilidad del registro.

Además, los items son completamente dependientes de la orden; no tienen sentido fuera de ella. Desde el punto de vista semántico y funcional, constituyen una parte integral del pedido. Por ello, su modelado como subdocumentos embebidos resulta coherente.

Aunque los items se almacenan embebidos para preservar el estado histórico del pedido, se incluye el campo articulo_id únicamente como identificador de referencia histórica. Este campo no se utiliza como fuente de verdad para reconstruir el pedido, sino como apoyo para análisis posteriores (por ejemplo, estadísticas de ventas por artículo), manteniendo la inmutabilidad de los datos relevantes como nombre y precio unitario.

**Referencing**

Por otro lado, se emplea el uso de referencias en los siguientes campos:

- ArticulosMenu.restaurante_id
- Ordenes.usuario_id
- Ordenes.restaurante_id
- Reseñas.usuario_id
- Reseñas.restaurante_id

Se adopta referencing en estos casos porque las entidades relacionadas (Usuarios, Restaurantes, Artículos de Menú y Órdenes) existen de manera independiente dentro del sistema. Estas entidades no dependen exclusivamente de un único documento, sino que pueden estar asociadas a múltiples registros.

Por ejemplo, un usuario puede generar múltiples órdenes y escribir varias reseñas; un restaurante puede tener numerosos artículos en su menú y recibir múltiples pedidos y reseñas. Si se optara por embedding en estos casos, se produciría una duplicación excesiva de información, lo cual afectaría la consistencia y aumentaría el tamaño de los documentos innecesariamente.

## Descripción de transacciones implementadas.

*Creación de un pedido*

1. Insertar documento en ordenes
2. Actualizar estado de artículos si aplica
3. Actualizar métricas del restaurante.
4. Actualizar Ordenes.estado constantemente.

Se utiliza una sesión con startTransaction() para asegurar atomicidad, evitando inconsistencias si falla la inserción del pedido y garantizando coherencia entre colecciones relacionadas

*Cancelación de un pedido*

1. Actualizar Ordenes.estado a “cancelado”
2. Revertir contadores o métricas acumuladas (si existen ventas acumuladas en restaurante).
3. Revertir disponibilidad de artículos al manejar stock.

Porque al cambiar el estado del pedido, no se actualizan las métricas, el sistema quedaría incoherente. Un restaurante podría aparecer con ventas que en realidad fueron canceladas.

*Eliminación de restaurante con limpieza referencial*

1. Eliminar restaurante
2. Eliminar o marcar como inactivos sus artículos de menú
3. Eliminar o archivar reseñas asociadas
4. Impedir nuevas órdenes

Sin transacción, se podría borrar el restaurante, pero dejar datos huérfanos.

## Explicación detallada de aggregation pipelines.

**Pipeline 1: Restaurantes mejor calificados**

1. **$group sobre Reseñas por restaurante_id**
2. **Calcular promedio con $avg**
3. **$sort descendente**
4. **$lookup con Restaurantes**
5. **$project campos relevantes**

**Pipeline 2: Platillos más vendidos**

- **$unwind de Ordenes.items**
- **$group por items.articulo_id**
- **$sum de cantidad**
- **$sort**
- **$lookup con ArticulosMenu**

**Pipeline 3: Total de ventas por restaurante por mes**

- **$group por:**
- **restaurante_id**
- **mes ($month)**

- **$sum del total**
- **$sort**

## Índices definidos y análisis de rendimiento con explain().

1. Índice simple
    a. db.usuarios.createIndex({ correo: 1}, {unique: true })
    Optimiza login y búsquedas por correo.
2. Índice compuesto
    a. db.ordenes.createIndex({ usuario_id: 1, fecha: -1 })
    Optimiza historial de pedidos por usuario ordenado por fecha.
3. Índice multikey
    a. db.restaurantes.createIndex({ categorias: 1 })
    Optimiza búsquedas por categoría.
4. Índice geoespacial
    a. db.restaurantes.createIndex({ ubicacion: "2dsphere" })
    Permite consultas por proximidad.
5. Índice de texto
    a. db.articulosMenu.createIndex({ nombre: "text", descripcion: "text" })
    Permite búsquedas textuales.

Validación con explain()

Utilizando el query *db.ordenes.find({ usuario_id: X }).explain("executionStats")*

Se verificará:

- IXSCAN en lugar de COLLSCAN
- Número reducido de documentos examinados
- Tiempo de ejecución

El sistema deberá rechazar consultas sin índice según requisito

## Propuesta de shard key y análisis de escalabilidad.

Como shard key, se propone el valor restaurante_id en colección Ordenes. Esto debido a su alta cardinalidad, distribución de carga por restaurante, evitaría hotspots si hay múltiples restaurantes activos y sería compatible con consultar frecuentes por restaurante.

Se prioriza restaurante_id como shard key debido a que las consultas analíticas principales (ventas por restaurante, platillos más vendidos y métricas comerciales) se ejecutan con base en restaurante. Aunque existen consultas por usuario (historial de

pedidos), estas representan un acceso más puntual y no cargas masivas de agregación, por lo que no justifican definir la shard key sobre usuario_id.

Respecto a las consideraciones que se deben tener para escalabilidad, consideramos lo siguiente.

- Separación de colecciones reduce documentos gigantes.
- Embedding controlado evita crecimiento ilimitado.
- Índices adecuados permiten crecimiento horizontal.
- Sharding distribuye carga en múltiples nodos.

El diseño es escalable porque:

- Las órdenes crecen sin afectar estructura de restaurante.
- Las reseñas se almacenan independientemente.
- Las consultas críticas están indexadas.

Enlace a la presentación

# ETAPA 02 - Elaboración del Backend

Link al GitHub